



S.G. Ganesh

# Parsing Complex Declarations in C

C is infamous for its complex (and clumsy?) declaration syntax. In this article, we'll look at a simple technique to parse such complex declarations in your head.

C (and of course, C++) is infamous for its complicated declaration syntax that invariably causes beginners to stumble. Complex declarations are hard to decode and understand. However, this is an essential area to master because you require it in real programming applications.

There is a simple traditional technique available called the 'clockwise' rule, which states, "Take any declaration, start with the innermost parenthesis (in the absence of parenthesis, start with the declared item's name), and work clockwise through the declaration going to the right first." For instance, with...

```
void (*arr[10]) ( );
```

...as the rule says, start from the innermost parenthesis and take only `(*arr[10])` to begin with. In that part, start with the variable name, which is 'arr'. Work clockwise and read it. "arr" is an array[10]. We hit ']', so go back and read \*, and so the sentence reads "arr" is an array[10] of pointer".

Then continue reading the remaining parts: "...to function returning type void". Reading it fully, it comes out that: "arr" is an array[10] of pointer to function returning type void". If you get confused about when to read left and when to go right, you can use this guideline: "Go to the right when you can, and to the left if you must". Simple, isn't it?

Just as an aside, this rule works for all declarations of original K&R C. But a slight change is required to accommodate the ANSI C addition of qualifiers like const, volatile and restrict. If they are used, read it together with the closest entity in the right-hand side. For example:

```
const int* (*foo)(void);
```

With this modified clockwise rule, we read it as: "foo is a pointer to function taking no arguments and returns a pointer to const int". Looks fine, right?

Enough with dummy examples; let's look at a real (and well-known) example. This declaration is from `<signal.h>`:

```
void (*signal(int sig, void (*func)(int)))(int);
```

Shall we parse it? "signal" is a pointer to the function taking two arguments 'sig' of int type and 'func', which is a pointer to a function taking int as an argument and returns void type; it returns a pointer to the function taking int as argument and returning void".

Let's accept that this is tough (even with all the help from the simple 'clockwise rule')! *typedef*'s can simplify complex declarations (see the Dec'08 JoP column). We'll use *typedef* to simplify this declaration:

```
typedef void (*sig_t) (int);
sig_t signal(int sig, sig_t func);
```

Yes, that's certainly more readable, isn't it? In general, follow this rule to create complex declarations: break the complex declarations into simpler entities and use *typedef*s for those small parts and combine them together. In this case, *typedef* is very handy: One reason is that the argument as well as the return type is *sig\_t*, so it looks very simple; but believe me, in reality, even with *typedef*s, some declarations can be hard to decipher!

Fortunately, there is a tool that you can use in Linux (and UNIX) when you want to understand complex declarations. It is called "cdecl" tool (c++decl for C++). You can use keywords like "explain", "declare", etc. This example is from the Linux man page for *cdecl(1)*: The input, "declare fptab" as an array of pointer to function returning pointer to char" will show the output, "char \*(\*fptab[])()". On the other hand, you can say "explain char \*(\*fptab[])()", to get that English-like description. That's handy. So, what are you waiting for – just go ahead, try out some complex declarations and have fun! 

## About the author:

**S G Ganesh** is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at [sgganesh@gmail.com](mailto:sgganesh@gmail.com).